

SmartLock

Présentation du projet :

- *Le projet SmartLock modernise la gestion du matériel du DeVinci Fablab en remplaçant les serrures traditionnelles par un système d'accès sécurisé et connecté via badge NFC.*
- *Cette solution couple la sécurisation physique des armoires à une gestion centralisée des stocks pour garantir une traçabilité complète des emprunts en temps réel.*
- *A terme, la solution sera implémentée toutes les armoires du Fablab afin de sécuriser le matériel de l'espace.*

VI - VF	Test Systeme	06/06/2026
Rédacteur : Florian Cazac	Relecteurs : Augustin Winterbert, Alexandre Houdin, Guilhem Henoux, Lois Halmaert, Yann Videmment	Jalon tests Systeme
	Validateurs : Florian Cazac	





1. Introduction

Ce document présente les tests système du projet SmartLock et leurs résultats finaux. Le projet SmartLock modernise la gestion du matériel du DeVinci FabLab en remplaçant les serrures traditionnelles par un système d'accès sécurisé par badge NFC, couplé à une gestion centralisée des stocks et à un tableau de bord d'administration.

1.1. Périmètre

Le plan couvre les sous-systèmes suivants et leur intégration :

- Interface Homme-Machine (IHM): application Python/CustomTkinter sur Raspberry Pi Zero 2W
- API d'authentification et d'autorisation : FastAPI + Keycloak + PostgreSQL
- Dashboard d'administration : SvelteKit / Node.js
- Matériel : lecteur NFC PN532, solénoïde 12V, relais GPIO, écran tactile 7" 1024×600
- Alimentation : 12V 3A secteur + convertisseur buck DC-DC

1.2. Légende des statuts

Statut	Signification
Validé	Test exécuté avec succès, critères de recette atteints
En cours	Test en cours ou sous-système partiellement validé
Bloqué	Test impossible - prérequis non satisfaits
Non commencé	Test planifié mais pas encore exécuté
Non implémenté	Fonctionnalité hors périmètre de la version livrée

2. Architecture système testée

Couche	Technologie	Hébergement
Physique	RPi Zero 2W + NFC PN532 + Relais GPIO26 + Solénoïde 12V + Écran 7"	Armoire FabLab
IHM	Python 3.13 / CustomTkinter	Raspberry Pi Zero 2W
API	FastAPI (Python) + Keycloak (OIDC) + PostgreSQL	Serveur DeVinci (Docker)
Dashboard	SvelteKit / Node.js	Serveur DeVinci (Docker)

2.1. Composants matériels

Composant	Spécification	Rôle
Raspberry Pi Zero 2W	ARM Cortex-A53, 512 MB RAM, WiFi	Exécute l'IHM, contrôle GPIO
Solénoïde 12V	12V DC, relais sur GPIO 26 (BCM)	Mécanisme de verrouillage
Alimentation	12V 3A secteur	Source principale du système
Écran tactile 7"	Waveshare HDMI 1024×600, capacitif	Interface kiosque utilisateur
Convertisseur buck	12V → 5V DC-DC	Alimentation RPi et écran
Lecteur NFC PN532	I ² C adresse 0x24, 13.56 MHz	Lecture des badges utilisateurs



3.3. Critères de recette par sous-système

3.1. IHM — Interface Homme-Machine

Responsable : Loïs Halmaert

Statut : **Validé**

- Démarrage automatique de l'IHM au boot (sans intervention manuelle)
- Lecture NFC détectée en < 2 secondes après présentation de la carte
- Toutes les vues fonctionnelles : home, navigation, sélection, panier, validation, accès, clôture
- Timer d'inactivité actif : 2 min sur home → fermeture, 90s sur autres vues → retour home
- Mode SIMULATION_MODE bascule sans erreur entre mode réel et simulé
- Connexion API confirmée (Keycloak client_credentials réussi)

3.2. API d'authentification et d'autorisation

Responsable : Florian Cazac

Statut : **Validé**

- 87/87 tests automatisés (pytest) passants sans échec
- Endpoint POST /auth/locker/{id}/check répond en < 500 ms en conditions nominales
- Authentification Keycloak opérationnelle (JWT valide, JWKS accessible)
- Service accounts smartlock-lockers et nfc-scanner fonctionnels
- Journal d'audit enregistre chaque tentative d'accès (accordée et refusée)
- Base PostgreSQL accessible, migrations appliquées (alembic upgrade head)
- API déployée via Docker Compose, accessible sur le réseau interne

3.3. Dashboard d'administration

Responsable : Yann Vidamment

Statut : **Validé**

- Authentification SSO Keycloak fonctionnelle (Authorization Code + PKCE)
- OTP requis pour les rôles T2+ (Bureau, Responsables)
- Gestion utilisateurs : liste, rôles, activation/révocation
- Gestion armoires : CRUD, matrice de permissions (rôle × armoire)
- Gestion stocks : vue globale, alertes stock bas, export CSV
- Journal d'audit consultable avec filtres, export CSV
- Application déployée via Docker, accessible sur le réseau interne (port 3000)



3.4. Lecteur NFC PN532

Responsable : Loïs Halmaert

Statut : **Validé**

- Détecté sur le bus I²C à l'adresse 0x24 (vérifié via `i2cdetect -y 1`)
- Badge détecté en moins de 2 secondes après présentation
- UID de carte transmis correctement à l'API

3.5. Solénoïde et relais GPIO

Responsable : Loïs Halmaert / Guilhem Henoux

Statut : **Validé**

- Relais GPIO 26 (BCM) activé depuis Python (RPi.GPIO)
- Activation pendant 5 secondes puis désactivation automatique
- Clic audible et tension 12V mesurée sur le solénoïde lors de l'activation

3.6. Batterie de secours (LiPo + BMS)

Responsable : Guilhem Henoux

Statut : **Non implémenté**

- Composants R_SC en attente de livraison (Mouser/LCSC)
- PCB prévu - emplacements disponibles, intégration non réalisée
- Basculement automatique sur batterie : NON TESTÉ
- Autonomie ≥ 30 min sous charge nominale : NON TESTÉ

3.7. Capteur à effet Hall

Responsable : Guilhem Henoux / Loïs Halmaert

Statut : **Non implémenté**

- GPIO 27 réservé dans le code - capteur non câblé physiquement
- La détection de fermeture utilise un timer de 5 secondes en remplacement

3.8. SKSM

Responsable : —

Statut : **Non implémenté**

- Non inclus dans le périmètre de la version livrée
- À définir et implémenter dans une version future



4. Matrice de tests système

Résultats finaux au 05/06/2026. Chaque test est identifié par un code SYS-XX.

ID	Test	Sous-systèmes	Responsable	Prérequis	Statut	Observations
SYS-01	Démarrage complet sous-alimentation secteur (RPi + écran)	Buck DC-DC, RPi	Guilhem / Loïs	Buck 12V→5V OK	Validé	Testé avec alim directe 5V
SYS-02	Démarrage complet sur batterie de secours	PCB, BMS, Buck	Guilhem	SYS-01 + BMS	Non impl.	Batterie non intégrée
SYS-03	Affichage IHM sur écran 7" Waveshare HDMI	Buck, RPi, IHM	Florian	SYS-01	Validé	Rotation portrait 90°
SYS-04	Lecture NFC et identification utilisateur via API	IHM, RPi, API	Loïs / Florian	SYS-03	Validé	Testé réel + SIMULATION
SYS-05	Ouverture casier sur badgeage autorisé (solénoïde)	IHM, API, relais	Florian / Loïs	SYS-04	Validé	Relais GPIO26 activé 5s
SYS-06	Refus d'accès — carte NFC inconnue	IHM, API	Florian	SYS-04	Validé	card_not_registered
SYS-07	Refus d'accès — compte révoqué	IHM, API	Florian	SYS-04	Validé	account_revoked
SYS-08	Refus d'accès — absence de permission	IHM, API	Florian	SYS-04	Validé	no_permission
SYS-09	Refus d'accès — permission expirée	IHM, API	Florian	SYS-04	Validé	expired
SYS-10	Détection fermeture casier (capteur Hall)	PCB, RPi, IHM	Guilhem / Loïs	SYS-05	Non impl.	Capteur non câblé
SYS-11	Journal d'audit enregistré après chaque accès	API	Florian	SYS-05	Validé	Vérifié via GET /logs
SYS-12	Comportement dégradé lors d'une coupure API	IHM, RPi	Loïs	SYS-03	Non commencé	Timeout + message erreur



SYS-13	Alerte Discord après 10 min de casier ouvert	IHM	Loïs	SYS-05	Non impl.	timer_manager vérifié
SYS-14	Basculement automatique sur batterie	PCB, BMS	Guilhem	SYS-02	Non impl.	Batterie intégrée non

5. Procédures de test pas-à-pas

5.1. Prérequis techniques

Variables d'environnement (adapter selon votre .env) :

```
API_URL="https://api.smartlock.devinci-fablab.fr"
KC_URL="https://auth.devinci-fablab.fr"
REALM="dev"      # ou prod
LOCKER_ID=1      # identifiant du casier cible
```

Obtenir un token RPi (client smartlock-lockers, grant client_credentials) :

```
RPI_TOKEN=$(curl -s -X POST "$KC_URL/realms/$REALM/protocol/openid-connect/token" \
-d "grant_type=client_credentials" -d "client_id=smartlock-lockers" \
-d "client_secret=$LOCKER_CLIENT_SECRET" | jq -r .access_token)
```

5.2. Procédure A — Accès complet nominal (SYS-04, SYS-05, SYS-11)

- Mise sous tension** : Connecter l'alimentation 12V 3A. Vérifier la tension 5V sur le RPi (4.8—5.2V au multimètre). Attendre le démarrage complet (~45s).
- Vérification réseau et API** : Tester : GET <https://api.smartlock.devinci-fablab.fr/health> → réponse attendue HTTP 200 {"status":"ok"}.
- Test NFC et accès casier** : Présenter une carte NFC enregistrée devant le lecteur PN532 (< 5 cm). Observer l'IHM : nom affiché en < 2s, clic du relais, ouverture du casier.
- Vérification journal d'audit** : GET /logs avec token admin → vérifier : allowed=true, user_id correct, timestamp récent.
- Fermeture et retour accueil** : Fermer le casier. L'IHM doit revenir à l'écran d'accueil automatiquement.

5.3. Procédure B — Tests de refus d'accès (SYS-06 à SYS-09)

- B1 — Carte inconnue** : Utiliser une carte NFC vierge. Résultat attendu : refus, log card_not_registered.
- B2 — Compte révoqué** : POST /users/{id}/revoke pour désactiver un compte test. Résultat attendu : refus, log account_revoked. Restaurer après test.
- B3 — Pas de permission** : Utiliser une carte d'un utilisateur sans permission sur le casier. Résultat attendu : refus, log no_permission.
- B4 — Permission expirée** : Configurer valid_until dans le passé. Résultat attendu : refus, log expired.



5.4. Procédure C — Test de robustesse (25 badgeages consécutifs)

- Système sous tension depuis au moins 10 minutes (température stabilisée)
- Effectuer 25 badgeages consécutifs, 10 secondes d'intervalle, casier refermé entre chaque
- Vérification : GET /logs → 25 entrées allowed=true
- Aucune erreur dans docker compose logs api
- Critères de succès : 25/25 badgeages réussis, aucune erreur, températures stables

6. Fonctionnalités non implémentées

Les fonctionnalités suivantes ont été identifiées dans la conception mais n'ont pas pu être réalisées dans la version livrée.

6.1. Batterie de secours (LiPo 3S + BMS)

Motif : Les résistances de protection R_SC nécessaires au régulateur MC34063ADR n'ont pas été livrées à temps (commande Mouser/LCSC en attente). Le PCB est prévu pour cette intégration.

Plan d'action : Intégrer les R_SC dès réception, valider les tensions, puis tester le basculement (SYS-14).

6.2. Capteur à effet Hall (détection fermeture casier)

Motif : Le capteur n'a pas été câblé physiquement. Le GPIO 27 est réservé et le code est en place.

Plan d'action : Câbler le capteur (VCC=3.3V, GND, OUT=GPIO27) — aucune modification logicielle nécessaire.

6.3. SKSM

Motif : Non défini dans le périmètre final du projet.

Plan d'action : À spécifier et implémenter dans une version future.